

Redes Neuronales Bien Entrenadas

Sprint 10 · Una guía completa desde la neurona biológica hasta los optimizadores modernos. Aprenderás a construir, entrenar y depurar redes neuronales con sólidas bases matemáticas e intuición práctica.

SPRINT 10 · S10.1

Neurona biológica vs. neurona artificial

🧠 Neurona biológica

- **Dendritas**: reciben señales de otras neuronas
- **Soma**: integra las señales entrantes
- **Axón**: transmite la señal de salida
- **Sinapsis**: conexión con la siguiente neurona; su fuerza varía con el aprendizaje
- La neurona «dispara» solo si el estímulo supera un umbral

🤖 Neurona artificial

- **Entradas $x_1 \ldots x_n$**: valores numéricos (píxeles, características, etc.)
- **Pesos $w_1 \ldots w_n$**: análogos a la fuerza sináptica
- **Sesgo b**: desplaza el umbral de activación
- **Combinación lineal**: $z = \Sigma\, w_i x_i + b$
- **Función de activación f(z)**: decide si y cómo «dispara»

La analogía biológica es útil como motivación, pero la neurona artificial es en esencia un **regresor lineal con no-linealidad aplicada encima**.

El Perceptrón

El perceptrón, propuesto por Frank Rosenblatt en 1958, es el modelo más simple de neurona artificial. Es el ancestro directo de todas las redes profundas modernas.

Funcionamiento

Dado un vector de entrada **x**, el perceptrón calcula una suma ponderada y aplica una función de activación escalonada:

$$\hat{y} = \text{sign}\left(\sum_i w_i x_i + b\right)$$

Si la salida es correcta, los pesos no cambian. Si no, se actualizan proporcionalmente al error. Este proceso se repite hasta convergencia.

Entradas

$x_1, x_2, \ldots, x_n$ — valores reales

Pesos

$w_1, w_2, \ldots, w_n$ — parámetros aprendidos

Salida

$\hat{y} \in \{-1, +1\}$ — clasificación binaria

Limitación histórica

Minsky y Papert (1969) demostraron que el perceptrón simple **no puede aprender la función XOR**, lo que desaceleró la investigación durante años. La solución: apilar capas.

Funciones de Activación

Las funciones de activación introducen **no-linealidad** en la red. Sin ellas, apilar capas sería equivalente a una única transformación lineal, incapaz de aproximar funciones complejas.



Sigmoid σ(z)

Salida en (0,1). Útil para probabilidades, pero sufre *vanishing gradient* en redes profundas.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Tanh(z)

Salida en (−1,1), centrada en cero. Mejor que sigmoid para capas ocultas, pero sigue teniendo vanishing gradient.

$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



ReLU(z)

La más usada en la práctica. Simple, eficiente y no sufre vanishing gradient para valores positivos.

$$\mathrm{ReLU}(z) = \max(0, z)$$



Softmax

Convierte un vector de scores en probabilidades que suman 1. Estándar en la capa de salida para clasificación multiclase.

$$\mathrm{softmax}(z_j) = \frac{e^{z_j}}{\sum_k e^{z_k}}$$

Anatomía de una red neuronal multicapa

Capa de entrada

Una neurona por cada característica del dato. No aplica transformación: solo distribuye los valores.

Capas ocultas

Donde ocurre el aprendizaje real. Cada capa extrae representaciones más abstractas que la anterior.

Capa de salida

Produce la predicción final. Su diseño depende de la tarea: 1 neurona (regresión), k neuronas + softmax (clasificación).

¿Cuántas capas y neuronas?

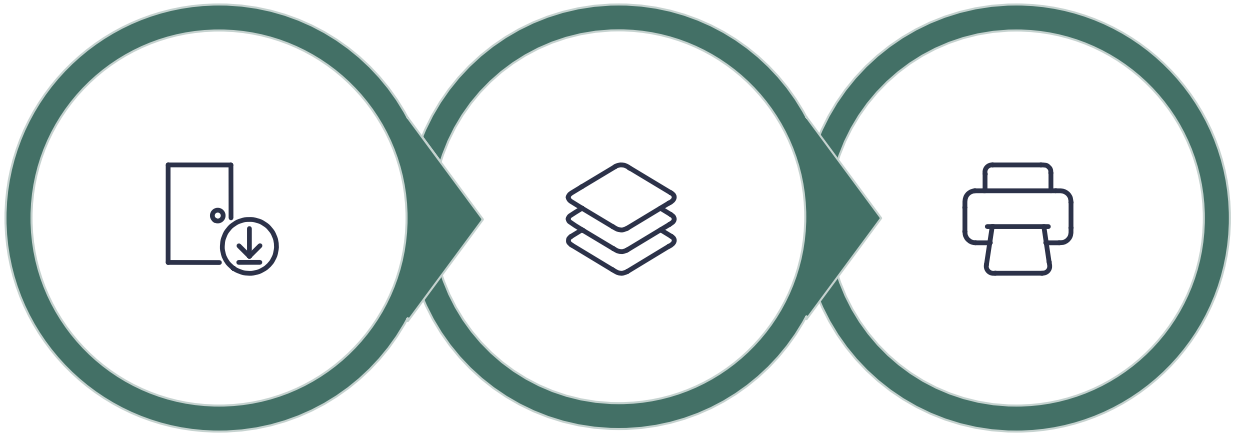
- Más capas → mayor capacidad de abstracción
- Más neuronas por capa → mayor capacidad por nivel
- Demasiada capacidad → riesgo de sobreajuste

Notación compacta

Una red **[784 → 128 → 64 → 10]** tiene tres capas con pesos, apropiada para clasificar dígitos MNIST.

El Forward Pass

El *forward pass* (pasada hacia delante) es el proceso mediante el cual un dato de entrada recorre la red capa a capa hasta producir una predicción. Es determinista: dados los pesos actuales, el resultado es siempre el mismo.



Para cada capa ℓ se calcula: $\mathbf{z^{(l)} = W^{(l)} \cdot a^{(l-1)} + b^{(l)}}$ y luego $\mathbf{a^{(l)} = f(z^{(l)})}$. La salida de cada capa es la entrada de la siguiente. Este proceso es eficiente y completamente paralelizable en GPU.

Función de Pérdida

La función de pérdida (o *loss function*) cuantifica cuán lejos está la predicción ŷ de la etiqueta real y. Es la brújula que guía el entrenamiento: el objetivo es **minimizarla**.

Regresión: MSE

Error cuadrático medio. Penaliza errores grandes de forma cuadrática, lo que hace que la red sea sensible a outliers.

$$\mathcal{L}_{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Clasificación: Cross-Entropy

Mide la divergencia entre la distribución predicha (softmax) y la distribución real (one-hot). Es la elección estándar en clasificación.

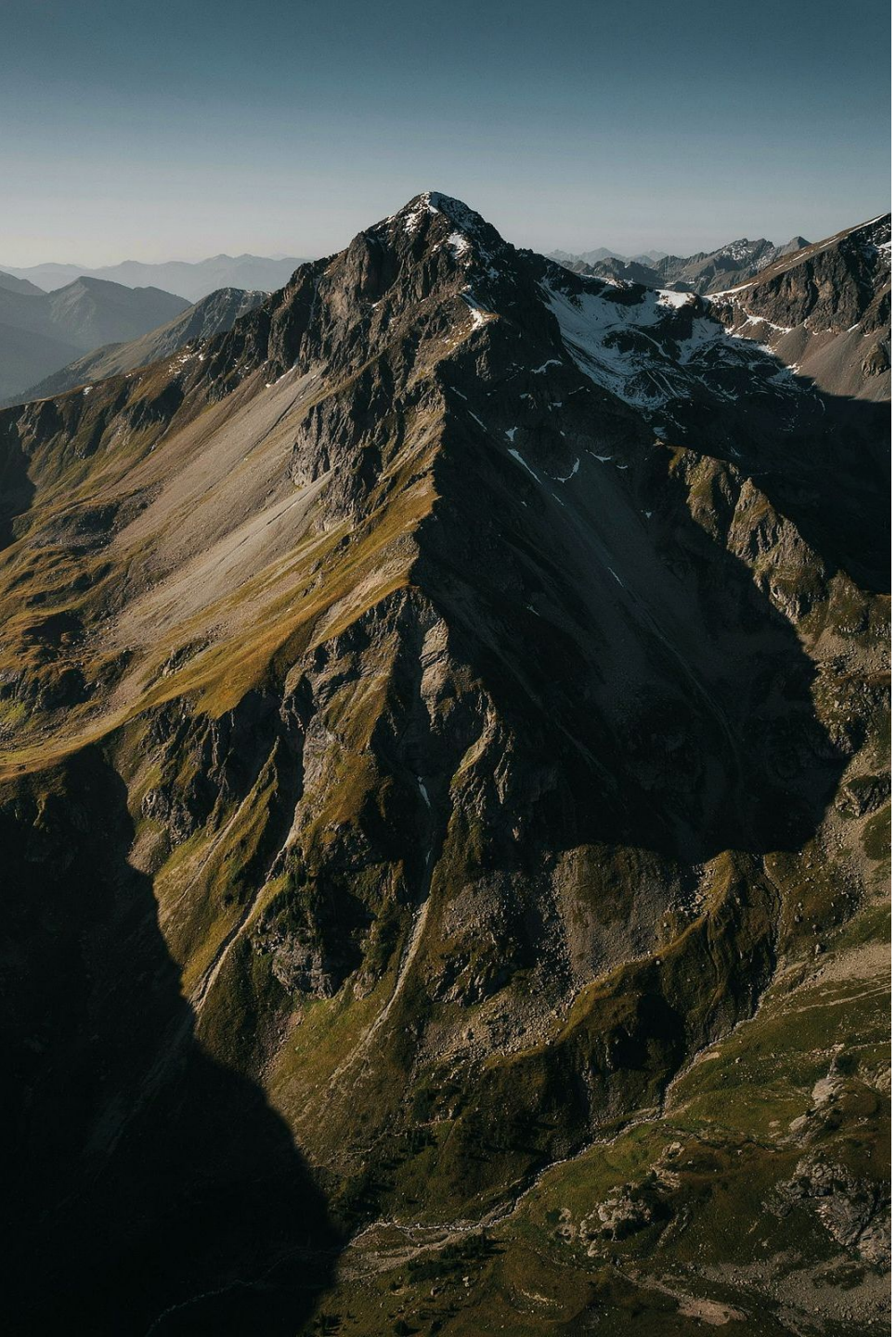
$$\mathcal{L}_{CE} = -\sum_{c} y_c \log(\hat{y}_c)$$

La función de pérdida define implícitamente qué tipo de tarea estás resolviendo. Cambiarla cambia el comportamiento del entrenamiento de forma fundamental.

Backpropagation

El algoritmo que hace posible el aprendizaje profundo: calcular el gradiente de la pérdida respecto a cada peso de la red.





Optimizadores

Los optimizadores definen cómo se actualiza cada peso a partir del gradiente. Son el motor del entrenamiento.

Adam: Adaptive Moment Estimation

Adam (Kingma & Ba, 2015) combina las ideas de momento y de tasas de aprendizaje adaptativas por parámetro. Es el optimizador por defecto en la mayoría de proyectos de deep learning.

Algoritmo en 3 pasos

1. Primer momento (media exponencial de gradientes):

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla \mathcal{L}$$

2. Segundo momento (media de gradientes al cuadrado):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)(\nabla \mathcal{L})^2$$

3. Actualización con corrección de sesgo:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\hat{m}_t$$

Hiperparámetros por defecto

$\eta = 0.001$ Learning rate inicial	$\beta_1 = 0.9$ Decaimiento del primer momento
$\beta_2 = 0.999$ Decaimiento del segundo momento	$\epsilon = 1e{-}8$ Estabilidad numérica

Ejemplo en NumPy: Red Neuronal desde Cero

Implementemos una red **[2 → 4 → 1]** para resolver XOR, el problema que derrotó al perceptrón simple. Toda la magia en menos de 30 líneas.

```
import numpy as np

# Activaciones
def sigmoid(z): return 1/(1+np.exp(-z))
def sigmoid_grad(a): return a*(1-a)

# Datos: XOR
X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([[0],[1],[1],[0]])

# Inicialización de pesos
np.random.seed(42)
W1 = np.random.randn(2,4)*0.1
b1 = np.zeros((1,4))
W2 = np.random.randn(4,1)*0.1
b2 = np.zeros((1,1))
```

```
lr = 0.5
for epoch in range(10000):
  # Forward
  z1 = X @ W1 + b1
  a1 = sigmoid(z1)
  z2 = a1 @ W2 + b2
  a2 = sigmoid(z2)
  # Pérdida MSE
  loss = np.mean((y - a2)**2)
  # Backward
  dL_da2 = -2*(y-a2)/len(y)
  dL_dz2 = dL_da2 * sigmoid_grad(a2)
  dL_dW2 = a1.T @ dL_dz2
  dL_da1 = dL_dz2 @ W2.T
  dL_dz1 = dL_da1 * sigmoid_grad(a1)
  dL_dW1 = X.T @ dL_dz1
  # Actualización
  W2 -= lr * dL_dW2
  W1 -= lr * dL_dW1
  b2 -= lr * dL_dz2.sum(0)
  b1 -= lr * dL_dz1.sum(0)

print(a2.round(2))  # [[0.02],[0.97],[0.97],[0.03]]
```

Puntos clave del código

Inicialización pequeña

Pesos ×0.1 para evitar saturación de sigmoid desde el inicio.

Forward almacena a1

Las activaciones intermedias son necesarias en el backward: no las descartes.

Gradientes = álgebra matricial

Toda la backprop son productos de matrices y multiplicaciones elemento a elemento.

Sin frameworks

Entender esto es el requisito previo para usar PyTorch o TensorFlow con criterio.

S10 · EN EL REPOSITORIO DEL CURSO

Ahora, a practicar

Lo que viene a continuación está en tu repositorio; no hace falta buscar ejercicios fuera.

CUADERNO

S10_01_red_desde_cero — 40 min

Una red neuronal con solo NumPy

sprints/S10/notebooks/S10_01_red_desde_cero.ipynb